

10.7 Rank-revealing factorizations

A model matrix with an intercept and indicators for every level of a factor has dependent, or **aliased**, columns. Software must notice them and decide which to set aside, but in floating point it can only ask whether a column is *nearly* a combination of others, relative to a tolerance. The SVD answers that (Section 10.4) but does not say which columns to drop, and it costs more. The standard tool is QR with column pivoting.

10.7.1 QR with column pivoting

At step k of Householder QR, the trailing columns (rows k to n , columns k to p of the partly reduced matrix) are what remains of the original columns after the parts explained by the first $k - 1$ chosen columns have been removed. **Column pivoting** (Businger and Golub 1965) moves the trailing column of largest norm, the one least explained by the columns already chosen, into position k before reflecting. The result is

$$\mathbf{X}\mathbf{\Pi} = \mathbf{Q} \begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix}, \quad \mathbf{R} = \begin{pmatrix} \mathbf{R}_{11} & \mathbf{R}_{12} \\ \mathbf{0} & \mathbf{R}_{22} \end{pmatrix},$$

with $\mathbf{\Pi}$ a permutation matrix, $\mathbf{R}_{11}$ the leading $k \times k$ block and $\mathbf{R}_{22}$ the trailing $(p - k) \times (p - k)$ block, for any k . We need one fact about submatrices.

Lemma 10.7.1 — Singular values of a submatrix

If $\mathbf{X}_1$ is formed from some of the columns of $\mathbf{X}$, then $\sigma_i(\mathbf{X}_1) \leq \sigma_i(\mathbf{X})$ for every i up to the number of columns of $\mathbf{X}_1$. The same holds for a submatrix formed from some of the rows.

Proof. Write $\mathbf{X}_1 = \mathbf{X}\mathbf{S}$, where $\mathbf{S}$ holds the corresponding columns of the identity, so $\|\mathbf{S}\|_2 = 1$. Let $\mathbf{X}_{i-1}$ be the truncated SVD with $i - 1$ terms, or $\mathbf{X}$ itself if $\text{rank } \mathbf{X} < i$, so that $\|\mathbf{X} - \mathbf{X}_{i-1}\|_2 = \sigma_i(\mathbf{X})$. The matrix $\mathbf{X}_{i-1}\mathbf{S}$ has rank at most $i - 1$, so by the Eckart–Young inequality, in the form used in the proof of Proposition 10.4.2,

$$\sigma_i(\mathbf{X}_1) \leq \|\mathbf{X}\mathbf{S} - \mathbf{X}_{i-1}\mathbf{S}\|_2 \leq \|\mathbf{X} - \mathbf{X}_{i-1}\|_2 \|\mathbf{S}\|_2 = \sigma_i(\mathbf{X}).$$

For rows, apply this to $\mathbf{X}^\top$, which has the same singular values. ■

Theorem 10.7.2 — Column-pivoted QR

Let $\mathbf{X}$ be $n \times p$ with $n \geq p$, and let $\mathbf{X}\mathbf{\Pi} = \mathbf{Q} \begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix}$ be computed by Householder QR (Theorem 10.3.2) with column pivoting.

- (a) For every $k \leq j$, $r_{kk}^2 \geq \sum_{i=k}^j r_{ij}^2$. In particular $|r_{11}| \geq |r_{22}| \geq \cdots \geq |r_{pp}|$.
- (b) For every $k < p$,

$$\sigma_{k+1}(\mathbf{X}) \leq \|\mathbf{R}_{22}\|_2 \leq \sqrt{p - k} |r_{k+1,k+1}| \quad \text{and} \quad \sigma_{\min}(\mathbf{R}_{11}) \leq \sigma_k(\mathbf{X}).$$

- (c) In exact arithmetic, if $\text{rank } \mathbf{X} = r$, then $r_{kk} \neq 0$ for $k \leq r$ and $\mathbf{R}_{22} = \mathbf{0}$ for $k = r$. The first r columns of $\mathbf{X}\mathbf{\Pi}$ are a basis of $\mathcal{C}(\mathbf{X})$, and $\mathbf{X}\mathbf{\Pi} = \mathbf{Q} \begin{pmatrix} \mathbf{R}_{11} & \mathbf{R}_{12} \\ \mathbf{0} & \mathbf{0} \end{pmatrix}$ with $\mathbf{R}_{11}$ nonsingular.
- (d) The cost is that of the unpivoted factorization, $2np^2 - 2p^3/3$ flops, plus $O(np)$ for keeping track of the column norms.

Proof.

- (a) Reflections preserve the length of each column segment they act on. When column k is chosen, its segment in rows $k, \dots, n$ has the largest norm among the trailing columns, and the reflection maps it to $(r_{kk}, 0, \dots, 0)^\top$, so $|r_{kk}|$ equals that norm. Every column that ends in a position $j > k$ was among the trailing columns at step k , so its segment in rows $k, \dots, n$ had norm at most $|r_{kk}|$. The reflections at steps $k, \dots, p$ act on rows $k, \dots, n$ only, so they preserve this norm, and at the end the segment is $(r_{kj}, \dots, r_{jj}, 0, \dots, 0)^\top$. Hence $\sum_{i=k}^j r_{ij}^2 \leq r_{kk}^2$. With $j = k+1$ this gives $r_{k+1,k+1}^2 \leq r_{kk}^2$.
- (b) $\mathbf{X}\mathbf{\Pi}$ has the same singular values as $\mathbf{X}$, because $\mathbf{\Pi}$ is orthogonal. The matrix $\mathbf{Q} \begin{pmatrix} \mathbf{R}_{11} & \mathbf{R}_{12} \\ \mathbf{0} & \mathbf{0} \end{pmatrix}$ has rank at most k and differs from $\mathbf{X}\mathbf{\Pi}$ by $\mathbf{Q} \begin{pmatrix} \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{R}_{22} \end{pmatrix}$, whose norm is $\|\mathbf{R}_{22}\|_2$. The Eckart–Young inequality (Proposition 1.8.2(c)) gives $\sigma_{k+1}(\mathbf{X}) \leq \|\mathbf{R}_{22}\|_2$. Next, $\|\mathbf{R}_{22}\|_2 \leq \|\mathbf{R}_{22}\|_F \leq \sqrt{p-k} \max_j \|\text{column } j \text{ of } \mathbf{R}_{22}\|$, and by (a) with $k+1$ in place of k , each column of $\mathbf{R}_{22}$ has norm at most $|r_{k+1,k+1}|$. Finally, the first k columns of $\mathbf{X}\mathbf{\Pi}$ equal $\mathbf{Q}_1 \begin{pmatrix} \mathbf{R}_{11} \\ \mathbf{0} \end{pmatrix}$, so their singular values are those of $\mathbf{R}_{11}$. Then Lemma 10.7.1 gives $\sigma_k(\mathbf{R}_{11}) \leq \sigma_k(\mathbf{X})$.
- (c) Because $\mathbf{R}_{11}$ is triangular, $\text{rank } \mathbf{R} = k + \text{rank } \mathbf{R}_{22}$ whenever $r_{11}, \dots, r_{kk}$ are nonzero (subtract multiples of the first k columns from the others to clear $\mathbf{R}_{12}$). Also $\text{rank } \mathbf{R} = \text{rank } \mathbf{X} = r$. So while $k < r$, $\mathbf{R}_{22} \neq \mathbf{0}$, some trailing column is nonzero, and the next pivot $r_{k+1,k+1}$ is nonzero. At $k = r$, $\text{rank } \mathbf{R}_{22} = 0$. The first r columns of $\mathbf{X}\mathbf{\Pi}$ are $\mathbf{Q}_1$ times the independent columns $\begin{pmatrix} \mathbf{R}_{11} \\ \mathbf{0} \end{pmatrix}$, so they are independent, and there are $r = \dim \mathcal{C}(\mathbf{X})$ of them.
- (d) The norms of the trailing column segments are updated at each step by subtracting r_{kj}^2 , at $O(p)$ flops per step. When cancellation makes the update inaccurate, the norm is recomputed.

■

Part (b) is the rank-revealing property, and it is one-sided. A small pivot $|r_{k+1,k+1}|$ certifies a small singular value σ_{k+1} : pivoted QR never reports a near dependence that is not there. The converse can fail. A matrix can have a tiny singular value although no pivot is small.

Example. (Kahan's matrix)

Kahan (1966) gave the upper triangular matrix with diagonal $1, s, s^2, \dots, s^{m-1}$ and entries $-cs^{i-1}$ above the diagonal in row i , where $c^2 + s^2 = 1$. Its columns are already in pivoted order (a tiny perturbation of the column scaling breaks ties in favour of that order), so pivoted QR leaves it unchanged. With $m = 100$ and $c = 0.2$, the smallest diagonal entry is 0.133, yet the smallest singular value is 3.7×10^{-9} . No diagonal entry hints at this (Figure 10.7.1(a)). Such matrices are rare in practice. Chan (1987) gave the first rank-revealing QR algorithm; factorizations whose bounds in both directions are polynomial in p exist (Hong and Pan 1992) and can be computed efficiently (Gu

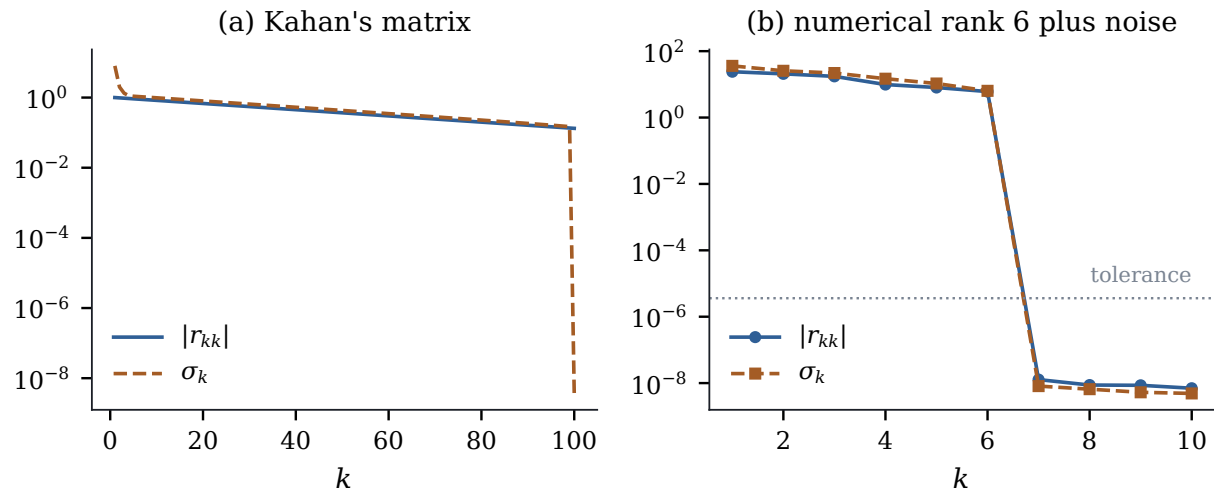


Figure 10.7.1: Diagonal of the pivoted triangle, $|r_{kk}|$, and singular values σ_k . (a) Kahan's matrix: the diagonal decreases slowly while the last singular value collapses. (b) A matrix of numerical rank six: both drop at $k = 7$, well below the tolerance.

and Eisenstat 1996). Panel (b) shows the typical case: a 50×10 matrix of rank six plus noise of relative size 10^{-9} . The pivoted diagonal drops by eight orders of magnitude exactly where the singular values do.

```
import numpy as np
import statsmodels.api as sm

def pivoted_qr(X):
    """Householder QR with column pivoting:  $X[:, \text{perm}] = Q R$ ,  $|r_{11}| \geq |r_{22}| \geq \dots$ """
    A = np.array(X, dtype=float)
    n, p = A.shape
    perm = np.arange(p)
    for k in range(min(n, p)):
        norms = np.sum(A[k:, k:] ** 2, axis=0)  # squared norms of the
        # trailing columns
        j = k + int(np.argmax(norms))
        A[:, [k, j]], perm[[k, j]] = A[:, [j, k]], perm[[j, k]]
        x = A[k:, k]
        alpha = -np.copysign(np.linalg.norm(x), x[0])
        if alpha == 0.0:
            break  # the trailing block is
        # exactly zero
        v = x.copy()
        v[0] -= alpha
        v /= np.linalg.norm(v)
        A[k:, k:] -= 2.0 * np.outer(v, v @ A[k:, k:])
    return np.triu(A[:p, :]), perm

def numerical_rank(R, tol):
    """Number of diagonal entries of the pivoted triangle above  $\text{tol} * |r_{11}|$ ."""
    d = np.abs(np.diag(R))
    return int(np.sum(d > tol * d[0]))
```

[Run this code online](#)

```

m, c = 100, 0.2
sn = np.sqrt(1 - c ** 2)
K = np.diag(sn ** np.arange(m)) @ (np.eye(m) - c * np.triu(np.ones((m, m)), 1))
K = K @ np.diag(1 - 1e-10 * np.arange(m))          # tiny tilt so that pivoting
                                                    keeps the order
RK, permK = pivoted_qr(K)
sK = np.linalg.svd(K, compute_uv=False)
print("pivot order unchanged:", np.array_equal(permK, np.arange(m)))
print(f"smallest |r_kk| = {np.abs(RK[-1, -1]):.3f}, smallest singular value = {sK
      [-1]:.2e}")

```

[Run this code online](#)

10.7.2 Basic solutions and aliased columns

Suppose pivoted QR finds numerical rank $r < p$, so that $\mathbf{X}\mathbf{\Pi} \approx \mathbf{Q} \begin{pmatrix} \mathbf{R}_{11} & \mathbf{R}_{12} \\ \mathbf{0} & \mathbf{0} \end{pmatrix}$. Write $\mathbf{Q}^\top \mathbf{y} = (\mathbf{c}_1^\top, \mathbf{c}_2^\top)^\top$ with $\mathbf{c}_1 \in \mathbb{R}^r$, and split the permuted coefficients as $(\mathbf{b}_1, \mathbf{b}_2)$. As in Proposition 10.3.3, the least squares estimates are the solutions of

$$\mathbf{R}_{11}\mathbf{b}_1 + \mathbf{R}_{12}\mathbf{b}_2 = \mathbf{c}_1,$$

with $\mathbf{b}_2$ free. Setting $\mathbf{b}_2 = \mathbf{0}$ gives the **basic solution**, which simply drops the last $p - r$ pivoted columns. It has at most r nonzero coefficients. It differs from the minimum-norm solution of Theorem 10.4.1. That solution can also be computed at QR cost, by a second orthogonal reduction from the right that turns $[\mathbf{R}_{11}, \mathbf{R}_{12}]$ into $[\mathbf{T}, \mathbf{0}]$ (a *complete orthogonal decomposition*; Golub and Van Loan 2013). Every least squares solution gives the same fitted values. By Theorem 6.4.4, and in the language of Chapter 8 (Theorem 8.3.1), every estimable function $\boldsymbol{\lambda}^\top \boldsymbol{\beta}$ has the same estimate under every convention. The coefficients of aliased columns are not estimable.

Software differs in the choice. R's `lm` takes the columns in order, moves to the end any column whose remaining norm falls below a relative tolerance (by default 10^{-7}), and reports its coefficient as `NA`. statsmodels' OLS uses the pseudoinverse and so reports the minimum-norm solution. Programs built on the sweep operator (Proposition 10.2.3), such as SAS's GLM procedure, skip numerically zero pivots and set those coefficients to zero, another basic solution.

Example. (Firm effects in investment data)

Grunfeld's investment data ($n = 220$: 11 US firms over 20 years, public domain in statsmodels) are modelled with an intercept, the firm's market value and capital stock, and indicators for all 11 firms: $p = 14$ columns of rank 13. Pivoted QR with a tolerance of 10^{-10} finds rank 13 and leaves out the indicator for General Motors. Three conventions give three different intercepts: -50.67 for the minimum-norm solution (as statsmodels reports), -20.58 for the in-order rule that marks the last indicator as aliased (as R would), and -70.30 for the basic solution from full pivoting. They agree to every printed digit on everything estimable: the coefficient 0.1101 of market value, 0.3100 of capital, the difference -172.20 between the General Motors and US Steel effects, and the residual sum of squares 523719.

```

gr = sm.datasets.grunfeld.load_pandas().data
firms = list(dict.fromkeys(gr["firm"]))

```

```

D = np.column_stack([(gr["firm"] == f).to_numpy(float) for f in firms])  # 11
    indicators
y = gr["invest"].to_numpy()
X = np.column_stack([np.ones(len(y)), gr[["value", "capital"]], D])      # 14
    columns, rank 13
n, p = X.shape
R, perm = pivoted_qr(X)
r = numerical_rank(R, 1e-10)
print("p =", p, " numerical rank =", r, " column left out:", perm[r:])

```

[Run this code online](#)

```

def basic_solution(X, y, tol=1e-10):
    """Drop the columns that pivoted QR puts last; fit the rest (zeros for dropped
    columns)."""
    R, perm = pivoted_qr(X)
    r = numerical_rank(R, tol)
    b = np.zeros(X.shape[1])
    b[perm[:r]] = np.linalg.lstsq(X[:, perm[:r]], y, rcond=None)[0]
    return b

def in_order_solution(X, y, tol=1e-7):
    """Keep a column only if it is not (nearly) a combination of the columns kept
    before it."""
    kept = []
    for j in range(X.shape[1]):
        Z = X[:, kept + [j]]
        Q, R = np.linalg.qr(Z)
        if abs(R[-1, -1]) > tol * np.linalg.norm(X[:, j]):
            kept.append(j)
    b = np.full(X.shape[1], np.nan)  # aliased coefficients reported
    # as missing
    b[kept] = np.linalg.lstsq(X[:, kept], y, rcond=None)[0]
    return b

solutions = {
    "minimum norm (pinv)": np.linalg.pinv(X) @ y,
    "pivoted QR, basic": basic_solution(X, y),
    "in order, aliased = NA": in_order_solution(X, y),
}
for name, b in solutions.items():
    fitted = X @ np.nan_to_num(b)
    print(f"{name:24s} value {b[1]:.5f} capital {b[2]:.5f} const {b[0]:9.3f}"
          f" GM - US Steel {b[3] - b[4]:9.3f} SSE {np.sum((y - fitted) ** 2):.1f}"
          )

```

[Run this code online](#)

10.7.3 Choosing a tolerance

Exact aliasing, as in the example, is easy: the offending pivot is at the level of rounding error, far below any reasonable tolerance. The hard case is a column that is nearly, but not exactly, a combination of others. The numerical rank then depends on the tolerance, and so do the coefficients, dramatically.

Example. (A nearly aliased column)

Add to the Grunfeld regressors (intercept, value, capital) a fourth column equal to value plus capital plus noise of relative size 10^{-9} . Its smallest singular value, relative to the largest, is 4.0×10^{-10} . The default tolerance of `numpy.linalg.lstsq`, $\max(n, p) \epsilon_M$ relative to σ_1 , treats the matrix as having full rank. It returns coefficients as large as 8.1×10^5 , which spend one more degree of freedom to reduce the residual sum of squares from 1768678.4 to 1768252.5, by 0.02 per cent. With the tolerance set to 10^{-8} , the fourth column is treated as aliased, and the largest coefficient is 38.41.

```
rng = np.random.default_rng(108)
z = X[:, 1] + X[:, 2] + 1e-9 * np.linalg.norm(X[:, 1]) * rng.normal(size=n) / np.
    sqrt(n)
Xz = np.column_stack([X[:, :3], z])          # constant, value, capital, value +
    capital + tiny
s = np.linalg.svd(Xz, compute_uv=False)
print("relative singular values:", np.array2string(s / s[0], precision=2))
for tol in [None, 1e-8]:
    b, _, rank, _ = np.linalg.lstsq(Xz, y, rcond=tol)
    print(f"rcond = {tol}: rank {rank}, largest |b_j| = {np.abs(b).max():.3g}, "
          f" SSE = {np.sum((y - Xz @ b) ** 2):.1f}")
```

[Run this code online](#)

Neither answer is wrong as arithmetic. But a column that differs from a combination of the others only at the level of 10^{-9} of its size carries no information a real measurement could support. Three rules follow. Scale the columns to comparable lengths before deciding the rank (Proposition 10.5.2); R's relative test does this implicitly. Tie the tolerance to the accuracy of the data, not of the machine: regressors recorded to six significant figures cannot resolve near dependences below about 10^{-6} of the column lengths (Proposition 10.4.2, applied to the recording error). And look for a gap, as in Figure 10.7.1(b). Without one the rank decision is fragile, the data are showing collinearity (Chapter 26), and shrinkage is often a better response than a sharp cut-off (Chapter 27).

Idea. Pivoted QR brings the most independent columns to the front. A small trailing pivot proves a near dependence, and setting those coefficients aside changes nothing estimable. The tolerance that decides “small” is a statement about the accuracy of the data.

10.7.4 Exercises

10.7.4.1 A. Check your understanding

Exercise A1.

Apply column-pivoted QR (Theorem 10.7.2) by hand to the matrix with columns $\mathbf{x}_1 = (1, 1, 0, 0)^\top$, $\mathbf{x}_2 = (0, 0, 1, 1)^\top$ and $\mathbf{x}_3 = (1, 1, 1, 1)^\top$ (use Gram–Schmidt on the pivoted columns, which gives the same $|r_{ij}|$). Find the pivot order, $\mathbf{R}$, and the rank. Give the basic solution for $\mathbf{y} = (1, 2, 3, 6)^\top$.

Solution. The norms are $\sqrt{2}, \sqrt{2}, 2$, so $\mathbf{x}_3$ is chosen first, with $|r_{11}| = 2$ and $\mathbf{q}_1 = \mathbf{x}_3/2$. Then $\mathbf{q}_1^\top \mathbf{x}_1 = \mathbf{q}_1^\top \mathbf{x}_2 = 1$, and the remainders are $\mathbf{x}_1 - \mathbf{q}_1 = (\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}, -\frac{1}{2})^\top$ and $\mathbf{x}_2 - \mathbf{q}_1$, its negative. Both have norm 1, so the tie goes to $\mathbf{x}_1$, with $|r_{22}| = 1$. The remainder of $\mathbf{x}_2$ is then zero, so $r_{33} = 0$ and the rank is 2. The pivot order is $(3, 1, 2)$, and up to signs $\mathbf{R} = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{pmatrix}$. The basic solution regresses $\mathbf{y}$ on $\mathbf{x}_3$ and $\mathbf{x}_1$ with $b_2 = 0$. The fit is the pair of group means, 1.5 for the first two observations and 4.5 for the last two. So $b_3 = 4.5$ and $b_1 = 1.5 - 4.5 = -3$.

10.7.4.2 B. Practice

Exercise B1.

In the setting of this section, show that the basic solution and the minimum-norm solution have the same fitted values. Show that the minimum-norm solution is orthogonal to $\mathcal{N}(\mathbf{X})$, while the basic solution in general is not. For the one-way layout of Example (Four answers, one fit), which generalized inverse in that example gives the basic solution that drops the intercept?

Solution. Both satisfy $\mathbf{R}_{11}\mathbf{b}_1 + \mathbf{R}_{12}\mathbf{b}_2 = \mathbf{c}_1$, so both are least squares estimates (Proposition 10.3.3), and by Theorem 6.4.1 both have fitted values $\mathbf{M}\mathbf{y}$. The minimum-norm solution lies in $\mathcal{C}(\mathbf{X}^\top) = \mathcal{N}(\mathbf{X})^\perp$ (Theorem 10.4.1(b)). The basic solution has zeros in the positions of the dropped columns. A null vector with a nonzero entry in one of those positions is generally not orthogonal to it. For the one-way layout, dropping the intercept column and fitting the three indicators is the “drop intercept” generalized inverse of Example (Four answers, one fit), which gives the group means as coefficients and a zero intercept.

Exercise B2.

Show that $|r_{11}| = \max_j \|\mathbf{x}_j\|$, and deduce $|r_{11}| \leq \sigma_1(\mathbf{X}) \leq \sqrt{p}|r_{11}|$. Combine with Theorem 10.7.2(b) to show that $\kappa(\mathbf{X}) \geq |r_{11}/r_{pp}|$, so that a large ratio of pivots proves ill-conditioning.

10.7.4.3 C. Going deeper

Exercise C1.

For Kahan’s matrix $\mathbf{K}_m(c)$, find a vector $\mathbf{x}$ with $\|\mathbf{x}\| = 1$ and $\|\mathbf{K}_m\mathbf{x}\|$ exponentially small in m when c is fixed. (Solve $\mathbf{K}_m\mathbf{x} = \mathbf{e}_m s^{m-1}$ by back substitution and examine the growth of the entries.) Why does column pivoting not detect this?